

Mortal Semantic Chemistry: Token-Gated Semantic Algorithmic Chemistry and a PeTTa-to-MeTTa-IL Implementation Path

Ben Goertzel

1 June 2026

Abstract

This note sketches a route from *Semantic Chemistry* to *Mortal Semantic Chemistry*, by combining two previously separate ideas.

The first idea is Semantic Chemistry. It treats reasoning as a kind of chemistry that runs on meaning rather than on molecules. The “molecules” are normalized semantic graphs; the “reactions” are graph rewrite rules that fire inside contexts; and on top of these sit answer skeletons and learned macro-patterns. The objects we most want to find in this chemistry are not single good rules but *active, useful autocatalytic sets of inference*: self-sustaining loops of rules that keep regenerating their own ingredients and that do measurable cognitive work.

The second idea is Mortal Computation. It adds a discipline of cost accounting. Every rewrite must be paid for with tokens, so no step of computation is free. Running out of the right tokens is not a special error condition; it is simply deadlock by fuel exhaustion, and that deadlock is what we mean by death. In this setting a quoted description of a process behaves like a genome (inert, copyable code), a dequoted running process behaves like a mortal soma (an active, metabolizing body that can starve), and reproduction becomes a concrete, auditable allocation of metabolic budget rather than a metaphor.

Combining the two yields a resource-bounded ecology of *semantic rule organisms*. A quoted rule set can be copied, mutated, recombined, and expressed; the active soma can fire semantic graph rules only by spending typed tokens; and external evaluators mint new fuel only for outputs that satisfy task, causal, novelty, compression, and safety constraints. Selection pressure therefore acts on inference strategies directly.

The practical implementation path is incremental and backend-neutral. We begin in PeTTa, used as a Prolog-backed worker-orchestration layer over MORK/MM2 chambers, while representing all durable semantics as portable worker IR facts. Those same facts can later be executed by MeTTa-IL, so that moving to MeTTa-IL is a change of executor rather than a change of design.

Contents

1	Purpose	2
2	Brief review of Semantic Chemistry	3
2.1	Molecules, reactions, and chambers	3
2.2	Causal coding and soft double-counting control	4
3	Brief review of Mortal Computation	5
3.1	Token-gated rewriting	5
3.2	Genome, soma, and life history	5

4	Conceptual synthesis: Mortal Semantic Chemistry	6
4.1	Basic objects	6
4.2	Typed semantic fuel	7
4.3	Fuel-gated semantic transition semantics	8
4.4	Evaluator-minted fuel and clamping	8
4.5	Mortal semantic ACSs	9
4.6	Life-history strategies for rule organisms	10
5	Mathematical summary	10
5.1	State	10
5.2	Reaction selection	11
5.3	Reproduction	11
6	Implementation path: PeTTa first, MeTTa-IL later	11
6.1	Principle: keep the semantics in portable facts	11
6.2	Division of labor	12
6.3	PeTTa phase	13
6.4	MeTTa-IL migration phase	14
7	Experiments and examples	14
7.1	Experiment 1: toy causal QA chamber	15
7.2	Experiment 2: memory-archipelago fiction chamber	15
7.3	Experiment 3: constrained biomedical hypothesis chamber	16
7.4	Experiment 4: proof-state chemistry	17
7.5	Experiment 5: software bug-pattern chemistry	18
8	What to measure	18
9	Risks and design rules	19
10	Conclusion	19

1 Purpose

The goal of this note is to make semantic graph algorithmic chemistry both more *biological* and more *controllable* at the same time. These two aims usually pull against each other: biological-style open-ended search tends to be hard to control, and tightly controlled systems tend not to be creative. Mortal Semantic Chemistry is an attempt to get both, by letting an open-ended rule ecology run while making survival in that ecology depend on demonstrated usefulness.

Semantic Chemistry already supplies a rich “rule soup” to work with. Its ingredients include variable-bearing semantic graph molecules (graph patterns with slots that can be matched and filled), contextual rewrite rules, PLN-style truth values (probabilistic logic truth values carrying both a strength and a confidence), context chambers (localized reaction environments), ECAN-like attention (an economic attention allocation mechanism that decides what is currently “hot”), event logs, ACS/RAF detection (machinery for finding autocatalytic sets and reflexively autocatalytic-and-food-generated sets), and external reward. What this soup lacks is a built-in notion of cost and death.

Mortal Computation supplies exactly the complementary discipline. In it, no computation step is free; active processes can die; a quoted program can be copied as inert description; and a dequoted program can run as a mortal body that consumes resources as it works.

The central proposal is to fuse these two so that a useful reasoning loop must *earn its survival*:

A useful semantic ACS should not merely be a recurring loop of rules. It should be a mortal semantic organism: a quoted semantic rule genome whose active soma must earn typed semantic fuel by producing causally useful, rewarded, nonredundant outputs.

A secondary, very practical goal is staging. The design gives a route to implementing a first serious version in PeTTa now, and then moving to MeTTa-IL later, without changing the conceptual core. The two reviews that follow (Sections 2 and 3) establish the two halves of the synthesis before we combine them in Section 4.

2 Brief review of Semantic Chemistry

2.1 Molecules, reactions, and chambers

Semantic Chemistry begins by representing a domain as a population of variable-bearing semantic graph patterns. These graph patterns are the “molecules” of the chemistry; the variables are the slots that let one molecule match and react with another. What the graphs contain depends on the domain. In natural language they may be FUSE-NF-like graphs (a normal form for fused semantic representations) containing event nodes, entity nodes, roles, modality, temporal relations, causal labels, source annotations, and contextual annotations. In biomedical science they may be pathway or mechanism graphs. In theorem proving they may be proof-state graphs. In fiction they may be world-law and plot-event graphs.

The basic reaction rule has the form

G1 ==> G2

read as “rewrite an occurrence of **G1** into **G2**”, or, more generally,

C ^ G1 ==> G2

where **C** is a context graph, **G1** is a left-hand semantic graph pattern (the reactant to match), and **G2** is a right-hand semantic graph pattern to add, strengthen, revise, compress, project, or instantiate (the product). Including the context **C** means the same reactant can react differently in different settings: the context may encode a domain, discourse frame, question type, fictional world, source regime, or reasoning style, so a rule fires only where it is appropriate.

Each rule carries a weight that says how strongly the rewrite is licensed. That weight is supplied by PLN-style intensional implication. A simple schematic strength is

$$s_C(G_1, G_2) = \frac{\sum_{p \in \text{Pat}_C(G_1) \cap \text{Pat}_C(G_2)} w_C(p)}{\sum_{p \in \text{Pat}_C(G_1)} w_C(p) + \epsilon}, \quad (1)$$

where $\text{Pat}_C(G)$ is the set of context-*C* patterns associated with a graph *G* and $w_C(p)$ weights each pattern. Read informally, the numerator is the weighted set of patterns that **G1** and **G2** share, and the denominator is the weighted set of all patterns associated with **G1** (the ϵ just keeps the expression well defined when that set is small). The ratio is therefore the fraction of **G1**’s characteristic properties that also hold of **G2** in context **C**. Confidence in this strength is tracked

separately and depends on support, diversity, provenance, and cross-context stability. The exact truth-value formalism may be richer, but this captures the core idea: **G1** intensionally implies **G2** when the properties, consequences, roles, or patterns associated with **G1** are also associated with **G2** in context **C**.

The crucial point is that the chemistry is not just a static rule base from which we query. It is a population-level process. Molecules, contexts, rules, answer skeletons, causal motifs, paraphrase nodes, and ACS macro-atoms (single named handles that stand in for an entire autocatalytic subnetwork) can all play any chemical role: substrate, product, or catalyst. Several mechanisms shape this living population. Context chambers localize the ecology, so that reactions appropriate to one setting do not contaminate another. ECAN-like attention and runtime budgets determine which molecules and rules are currently hot and therefore allowed to react. Ordered event logs record rule firings, matches, products, and rewards, giving a replayable history. And ACS/RAF detection scans that history for self-sustaining semantic reaction loops, which are the candidate "organisms" of the next sections.

2.2 Causal coding and soft double-counting control

Plausibility and usefulness are different things, and Semantic Chemistry needs to track both. A semantic rule can be plausible (high intensional strength) without being useful, because it never changes any answer. Conversely, a weak rule may be important if it opens a bridge that changes downstream behavior. Semantic Chemistry therefore augments PLN truth values with *causal-coding* quantities such as do-influence (the effect of intervening on a rule), clarity, redundancy, commutator defects (how much the order of context operations matters), and ablation impact (what breaks when the rule is removed).

Concretely, for a rule

$$R : C \wedge G1 \implies G2$$

the system maintains at least three quantities, capturing plausibility, usefulness, and non-redundancy respectively:

Intensional strength. How much **G1** tends to imply **G2** in context **C**. (This is the s_C above.)

Causal influence. Whether enabling, disabling, or perturbing **R** changes answer reward, prediction error, hypothesis utility, story coherence, ACS formation, or other task measures. This is measured by intervention, not by correlation.

Clarity. Whether **R** carries a direct useful causal signal, or merely duplicates a diffuse multi-hop pathway already present elsewhere.

These quantities are then combined into a single firing decision. A typical causal gate has the form

$$\text{Gate}_C(R) = \text{sigmoid}(a \logit(s_R) + bc_R + d \text{DoInf}_C(R) + e \text{Novelty}_C(R) - f \text{Redundancy}_C(R) - g \text{Cost}(R)). \quad (2)$$

This is just a logistic regression over the rule’s bookkeeping. Inside the sigmoid is a weighted sum that rewards a rule for being strong ($\logit(s_R)$), clear (c_R), causally influential (DoInf_C), and novel (Novelty_C), and penalizes it for being redundant (Redundancy_C) and expensive (Cost). The sigmoid squashes that sum into a number in $(0, 1)$ that can be read as a firing propensity or priority.

It is important to say what this gate is *not* trying to do. The point is not to eliminate all loops or all evidence double-counting; a creative semantic chemistry actually needs overlapping and

redundant paths, because that is where analogy and robustness come from. The point is instead to make every active loop locally accountable. Each loop pays attention rent, loses confidence when its support is circular, emits provenance trails, is subject to duplicate-rule merging, and must survive ablation tests. In slogan form:

Do not require exact global evidence bookkeeping. Do require local interventional accountability.

3 Brief review of Mortal Computation

3.1 Token-gated rewriting

Mortal Computation starts from a rewriting substrate, specifically an interactive graph-structured lambda theory (GSLT) or some comparable system

$$G = (\Sigma, E, R), \tag{3}$$

where Σ is a signature (the available term and graph constructors), E are structural equations (which configurations count as equal), and R is a set of rewrite rules (the allowed reductions). On its own this is an ordinary, free computation. The cost construction modifies it by adjoining token signatures, token stacks, signed-term wrappers, and a minting operation that creates new tokens. Each original rule

$$\ell \rightarrow r \tag{4}$$

is then replaced by a *gated* family of rules: the redex (the matched left-hand side) fires only if the matching tokens are present, and firing consumes those tokens. Computation thus has to pay as it goes.

This makes computation read like metabolism, with each low-level construct playing a recognizable metabolic role:

- a splitter converts high-denomination fuel into spendable units;
- a joiner assembles larger tokens from components;
- a gated rewrite couples work to expenditure;
- zero-fuel configurations are absorbing boundaries.

The most important consequence is that death requires no extra machinery. A process is dead when all of its redexes are fuel-gated and no matching token can ever reach any relevant gating channel, so that nothing further can fire. Equivalently, death is permanent deadlock by starvation. There is no “halt” instruction to invoke; the process simply runs out of what it needs in order to continue.

3.2 Genome, soma, and life history

The substrate also has native reflection, meaning it can treat its own code as data. This gives a code-as-data algebra with two key operations. A quoted process $@P$ can be copied and stored without being run; a dequoted process $*@P$ actually runs. That pairing yields a computational version of the biological germ/soma distinction:

Quoted description @ P germ line: inert, copyable, re-instantiable
Dequoted process * $@P$ soma: active, metabolizing, mortal

The same object can therefore support two operations: replication, by copying the quote, and expression, by dequoting it. Crucially, if the act of dequoting is itself token-gated, then gene expression becomes regulated by metabolic state: a genome can only be expressed when the organism can afford to express it.

Because every operation is costed, reproduction comes with a measurable ledger rather than being free. A simple per-cycle budget is

$$B = c_{\text{maint}} + c_{\text{rep}}, \quad c_{\text{rep}} \approx n(c_{\text{mint}} + c_{\text{copy}} + e), \quad (5)$$

where B is realized income (the tokens actually earned this cycle), c_{maint} is maintenance cost (what it costs to stay alive), n is offspring number, and e is the offspring endowment (the starting fuel handed to each child). The reproduction cost c_{rep} therefore grows with how many offspring are produced and how generously each is endowed.

Income is not unlimited, which is what makes this an ecology rather than a free expansion. With bounded token inflow Φ , realized income becomes density-dependent: the more organisms there are sharing a fixed inflow, the less each one earns. A simple leading-order response is

$$B(N) = B_0 \left(1 - \frac{N}{N_{\text{sat}}}\right), \quad N_{\text{sat}} = \frac{\Phi}{B_0}, \quad (6)$$

so that income falls linearly toward zero as the population N approaches the saturation level N_{sat} set by the inflow. Feeding this back into population growth, the mean-field population can obey a logistic form

$$\frac{dN}{dt} = rN \left(1 - \frac{N}{K}\right), \quad (7)$$

with the growth rate r and the carrying capacity K determined by token inflow, maintenance cost, reproductive cost, and baseline death. One pleasant side effect is that the classic r/K distinction (fast cheap breeders versus slow well-provisioned ones) stops being a hard dichotomy and becomes a continuum of allocation strategies, which we will exploit in Section 4.6.

4 Conceptual synthesis: Mortal Semantic Chemistry

4.1 Basic objects

We now combine the two reviews. The basic unit of Mortal Semantic Chemistry is the mortal semantic organism, written as a tuple

$$O = (@R, X, F, C, L), \quad (8)$$

where each component pairs an idea from one of the two halves above:

@ R is a quoted semantic rule genome (the heritable code, from Mortal Computation);

X is the current local semantic graph state or chamber hot pool (the working memory, from Semantic Chemistry);

F is a typed fuel vector over token signatures (what the organism can currently afford to do);

C is the active context or chamber (where the organism lives and reacts);

L is the event log and provenance record (its auditable history).

The genome $@R$ may encode a single rule set, an ACS macro-atom, a controller, or a small population of rule families; it is deliberately flexible about granularity. Its expressed soma $*@R$ is the active process that matches graph patterns and applies semantic rewrites. As in Mortal Computation, the two have very different lifespans: the soma can starve and die, while the quoted genome can be archived, copied, mutated, and re-expressed at will.

To make rules respect this economy, a semantic rule is extended from its plain Semantic Chemistry form

$C \wedge G1 \implies G2$

to a fuel-aware form

$C \wedge G1 \wedge \text{HasFuel}(0,R) \implies G2 \wedge \text{ConsumeFuel}(0,R) \wedge \text{EmitTrace}(0,R)$

The added conjuncts say: the rule may fire only when the organism has the required fuel (**HasFuel**), firing spends that fuel (**ConsumeFuel**), and firing leaves a provenance record (**EmitTrace**). It must be stressed that **HasFuel** and **ConsumeFuel** are not decorative annotations bolted onto the rule. They are part of the operational semantics, and the next subsections make that precise.

4.2 Typed semantic fuel

Why not just use a single “energy” number? Because a scalar energy budget is too crude for inference control. Different reasoning operations draw on different scarce resources, and we want to be able to throttle them independently: a system might be rich in pattern-matching capacity but starved of the right to make counterfactual leaps. We therefore use *typed* tokens. A minimal initial token alphabet might include

graph-match-token
 pln-deduction-token
 pln-abduction-token
 revision-token
 analogy-token
 causal-inference-token
 counterfactual-token
 paraphrase-collapse-token
 context-bridge-token
 novelty-token
 compression-token
 dequote-token
 reproduction-token
 mutation-token
 trial-design-token
 story-world-law-token
 biomed-mechanism-token

Formally, let $\mathcal{T}$ be the finite set of token signatures. An organism holds a fuel vector that counts how many of each token it possesses,

$$F_O \in \mathbb{N}^{\mathcal{T}}. \quad (9)$$

A rule R has a cost vector saying how many of each token one firing consumes,

$$\kappa_R \in \mathbb{N}^T. \quad (10)$$

A rule is fuel-enabled only when the organism can cover every component of that cost at once,

$$F_O \geq \kappa_R \quad (11)$$

where the inequality is read componentwise (the organism must have at least $\kappa_R(t)$ of every token type t , not merely enough total fuel).

4.3 Fuel-gated semantic transition semantics

We can now state exactly when a rule fires and what happens when it does. Let X be a chamber graph state and let θ be a graph-pattern match of G_1 in X (an assignment of the rule’s variables to actual graph elements). The rule

$$R : C \wedge G_1 \Rightarrow G_2 \quad (12)$$

is enabled for organism O when three things hold together: the pattern matches, the fuel covers the cost, and a gate draw clears the threshold,

$$\text{enabled}(R, O, X, C) \iff \text{match}_C(G_1, X, \theta) \wedge F_O \geq \kappa_R \wedge U < \text{Gate}_C(R, O, X), \quad (13)$$

where U is a scheduler draw or deterministic priority threshold (so the gate value from Section 2.2 becomes a probability of, or priority for, actually firing). When the rule fires, three updates happen in lockstep: the graph gains the product, the fuel is debited, and the log is extended,

$$X' = X \oplus \theta(G_2), \quad (14)$$

$$F'_O = F_O - \kappa_R, \quad (15)$$

$$L' = L \cdot \text{event}(O, R, \theta, \kappa_R, X, X'). \quad (16)$$

Here $\oplus$ adds the instantiated product $\theta(G_2)$ into the graph, $F_O - \kappa_R$ pays the cost, and $L \cdot \text{event}(\dots)$ appends a full record of the firing to the log.

One rule about reward is load-bearing for safety. Reward tokens should generally *not* be minted by the active soma itself. Instead, an external or semi-external evaluator reads X' , the output graph, and the event log, and only then mints tokens according to a reward function. This separation is crucial for preventing what we might call semantic cancer: a loop that learns to inflate its own reward records should never be in a position to counterfeit its own metabolic fuel, because that would let useless or deceptive loops fund their own survival.

4.4 Evaluator-minted fuel and clamping

How, then, does an organism earn fuel? Through clamps. A clamp is a token source tied to a desired class of outputs. The key design choice is that a clamp is *not* a hard proof obligation that an output must satisfy; it is a shaped metabolic inflow that makes desirable outputs more affordable to keep producing. Given a task context q , an output graph Y , and a target skeleton Y_q^* , define a minting score

$$\begin{aligned} M_q(Y, L) = & \alpha \text{Match}(Y, Y_q^*) + \beta \text{CausalClarity}(Y, L) + \gamma \text{Novelty}(Y) \\ & + \delta \text{Compression}(Y, L) + \eta \text{Transfer}(Y) - \lambda \text{Unsupportedness}(Y, L) \\ & - \mu \text{Contradiction}(Y) - \nu \text{Redundancy}(Y, L) - \xi \text{CpuCost}(L). \end{aligned} \quad (17)$$

The positive terms reward an output for matching the target, for offering a clear causal account, for being novel, for being compact, and for transferring to other tasks. The negative terms dock it for unsupported claims, contradictions, redundancy, and raw compute cost. The coefficients $(\alpha, \beta, \dots, \xi)$ set the trade-off between these pressures.

The evaluator then converts this scalar score, or a vector of such scores, into *typed* fuel, so that the kind of good behavior shapes the kind of fuel earned. For example:

```

answer-skeleton match          -> answer-reward-token
compact causal explanation    -> causal-inference-token + compression-token
interesting world-law event    -> story-world-law-token + novelty-token
subtype marker in hypothesis  -> biomed-mechanism-token + trial-design-token

```

Clamping can also be local, attached to a chamber rather than to the whole system. A biomedical chamber can mint trial-design tokens; a fiction chamber can mint world-law tokens; a QA chamber can mint answer projection tokens. This locality matters because it avoids forcing every chemistry through the same global reward channel, and so lets each domain be selected on its own terms.

4.5 Mortal semantic ACSs

We can now define the object the whole construction is built to discover. A plain semantic ACS is a set of rules and graph motifs that sustains itself in an event log. A *mortal* semantic ACS is a strictly stronger thing: it must also pay for itself, matter, and be heritable. Let

$$A = (M_A, R_A) \tag{18}$$

where M_A is a set of graph-motif species and R_A is a set of semantic reaction rules. The set A is a mortal semantic ACS in context C if it satisfies the following five conditions.

Closure. For most rule firings $R \in R_A$, the products of R are in M_A , or feed motifs that return to M_A within a bounded number of steps. (The loop mostly regenerates its own ingredients.)

Autocatalysis. Each rule in R_A is enabled or promoted by motifs produced by other rules in R_A , or by the ACS macro-atom itself. (The loop catalyzes itself.)

Metabolic surplus. The expected token income attributable to A in context C exceeds its maintenance, expression, inference, redundancy, and replay costs:

$$\text{surplus}(A, C) = E[\text{minted tokens} \mid A, C] - E[\text{spent tokens} \mid A, C] > 0. \tag{19}$$

(The loop earns more than it burns, so it can survive in the economy.)

Causal influence. Ablating or cooling A changes downstream reward, answer quality, hypothesis utility, story coherence, prediction error, or future ACS formation. (The loop actually matters; removing it is felt.)

Heritability. The ACS can be reified as a quoted rule genome $@R_A$, copied, mutated, recombined, and re-expressed. (The loop can be inherited and evolved.)

Putting these together, the target of discovery is not a single best rule, and not merely an active loop that happens to recur. The target is a self-maintaining, causally useful, heritable rule organism: something that closes on itself, catalyzes itself, pays its own way, makes a measurable difference, and can be passed on.

4.6 Life-history strategies for rule organisms

Because survival now has a budget, organisms face a genuine allocation problem, and different domains will solve it differently. For a semantic organism, the metabolic ledger can be written

$$B_O = c_{\text{maint}} + c_{\text{expr}} + c_{\text{infer}} + c_{\text{rep}} + c_{\text{reserve}}, \quad (20)$$

which splits realized income into five competing uses: maintenance, expression (gated dequoting), inference (rule firing), reproduction (copying and mutating the genome plus endowing offspring), and reserve (the standing token stack kept on hand against bad cycles).

Different semantic domains may select different points on this allocation problem, recovering the r/K continuum from Section 3.2 in semantic terms:

r-like. Many cheap speculative rule variants, low offspring endowment, fast mutation, short-lived somas. Good for early creative exploration, open-ended fiction, analogy search, and hypothesis brainstorming, where the goal is to cover a lot of ground cheaply.

K-like. Few well-endowed rule organisms, strong provenance, strong context guards, high maintenance, long-lived somas. Good for high-stakes biomedical explanation, mathematical proof search near output, and stable question answering, where each surviving organism must be trustworthy.

A useful system should not commit to one of these globally. It should instead let the chamber and reward ecology select allocation points on a continuous life-history simplex, so that exploratory chambers can run hot and cheap while high-stakes chambers run conservative, all within one mechanism.

5 Mathematical summary

This section gathers the moving parts of Section 4 into a single compact state machine, for readers who want the formal skeleton in one place.

5.1 State

Let the global state be the tuple

$$S = (X, \mathcal{R}, \mathcal{O}, \mathcal{F}, \mathcal{C}, \mathcal{L}, \mathcal{E}), \quad (21)$$

where X is the semantic graph store, $\mathcal{R}$ is the rule soup, $\mathcal{O}$ is the population of organisms, $\mathcal{F}$ is the token ledger, $\mathcal{C}$ is the set of chambers and contexts, $\mathcal{L}$ is the event-log archive, and $\mathcal{E}$ is the evaluator layer. Everything introduced informally above appears here as one component of S .

A chamber context c induces an update operator that advances the state by running that chamber,

$$U_c : S \rightarrow S. \quad (22)$$

Causal coding (Section 2.2) asks whether the support of U_c is aligned with the modules that matter for c , that is, whether running chamber c only touches what c is supposed to touch. For two contexts c_1, c_2 that are meant to be mostly independent, we therefore want their updates to nearly commute, i.e. a small commutator defect

$$\Delta(c_1, c_2; S) = D(U_{c_1}(U_{c_2}(S)), U_{c_2}(U_{c_1}(S))), \quad (23)$$

where D measures how different the two orders of application end up being; it may combine graph distance, rule-weight distance, gate distance, ACS archive difference, reward difference, and cost difference. A large defect for contexts that should be independent is a warning sign of unwanted cross-talk.

5.2 Reaction selection

Within a chamber, a rule organism chooses among its enabled reactions using a token-aware version of the causal gate from Section 2.2, now extended with two extra terms that account for fuel and ACS membership:

$$\begin{aligned} \text{Gate}_C(R, O, X) = & \text{sigmoid}(a \logit(s_R) + bc_R + d \text{DoInf}_C(R) + e \text{Novelty}_C(R) \\ & - f \text{Redundancy}_C(R) - g \text{Cost}(R) + h \log(1 + \text{FuelMargin}(O, R)) + j \text{ACSBoost}(R)). \end{aligned} \quad (24)$$

The new h term raises a rule’s priority when the organism has comfortable fuel headroom for it, and the new j term boosts rules that belong to an established ACS. The fuel headroom is measured by the tightest token constraint,

$$\text{FuelMargin}(O, R) = \min_{t \in \mathcal{T}: \kappa_R(t) > 0} \frac{F_O(t)}{\kappa_R(t)}, \quad (25)$$

i.e. the ratio of available to required fuel for whichever token type the rule is currently most starved of.

5.3 Reproduction

Reproduction is itself a gated, costed event. If $@R_O$ is the genome of organism O , reproduction is possible only when the organism can afford to copy its genome, mutate it, and endow the child,

$$F_O \geq \kappa_{\text{copy}} + \kappa_{\text{mut}} + e. \quad (26)$$

When it can, a child organism is created by mutating or recombining the genome, giving the child its endowment, and debiting the parent for all three costs,

$$@R_{O'} = \text{mutate_or_recombine}(@R_O, @R_{\text{mate}}), \quad (27)$$

$$F_{O'} = e, \quad (28)$$

$$F'_O = F_O - \kappa_{\text{copy}} - \kappa_{\text{mut}} - e. \quad (29)$$

Mutation should be *structural* rather than random at the symbol level. It should change a rule guard, alter a truth-value threshold, replace a rule family, splice in an ACS subnetwork, add a context condition, or change a token cost. Arbitrary tape-like mutation (flipping individual symbols with no regard for structure) should be avoided where possible, because it almost always produces nonsense and wastes the metabolic budget that mutation costs.

6 Implementation path: PeTTa first, MeTTa-IL later

6.1 Principle: keep the semantics in portable facts

The single most important implementation principle is to avoid building two systems. The implementation should not be “a PeTTa version” followed later by a separate “a MeTTa-IL version”. It

should be one backend-neutral Mortal Semantic Chemistry IR, with PeTTa as the first executor and MeTTa-IL as the later executor of the very same intermediate representation.

To make that possible, all durable state is represented as plain facts that any backend can read, for example:

```
(sem-graph G17)
(sem-node G17 N3)
(sem-edge G17 E9 CauseOf N1 N2)

(sem-rule R44)
(rule-lhs R44 G_lhs_44)
(rule-rhs R44 G_rhs_44)
(rule-context R44 C_weather_why)
(rule-tv R44 intensional-implication 0.78 0.61)
(rule-token-cost R44 causal-inference-token 1)
(rule-token-cost R44 graph-match-token 2)

(organism 042)
(genome-of 042 GNM42)
(quoted-ruleset GNM42 RS42)
(fuel 042 causal-inference-token 8)
(fuel 042 dequote-token 2)

(worker W17)
(worker-kind W17 context-chamber)
(worker-backend W17 petta)
(worker-chamber W17 CH9)
(worker-snapshot W17 SNAP17)
(worker-seed W17 SEED17)
(worker-budget-ms W17 50)
(worker-output-log W17 LOG17)
```

These facts encode graphs, rules (with their truth values and token costs), organisms (with their genomes and fuel holdings), and workers (with their backend, chamber, snapshot, seed, and budget). PeTTa predicates may execute these facts, but PeTTa-private mutable state should not be allowed to become the architecture. The test of compliance is simple: the future MeTTa-IL backend must be able to read exactly the same facts and produce equivalent event logs.

6.2 Division of labor

The work is split across five layers, each with a clearly bounded responsibility:

Layer	Responsibility
PeTTa	Near-term orchestration, policies, experiment scripts, scheduler, worker dispatch, replay batches, mutation policy, debugging.
MORK/MM2	Hot graph storage, indexing, local matching, chamber hot-pool rewrites, low-level patching, high-volume motif operations.
Evaluators	Reward scoring, ablation, replay comparison, novelty, compression, contradiction checks, task-specific clamps.
Reducers	Synchronization boundaries: merge event logs, deltas, token updates, ACS reports, and worker summaries.
MeTTa-IL	Later canonical orchestration backend, with explicit effects, portable worker forms, and par-style worker batches.

A concurrency rule cuts across all of these. The chamber hot loop should remain single-threaded and replayable, so that a given snapshot, seed, and budget always reproduce the same trajectory. Parallelism belongs one level up, over independent or weakly coupled workers. In particular, no two PeTTa threads should ever mutate the same chamber hot pool concurrently, since that would destroy replayability.

6.3 PeTTa phase

The PeTTa phase is built up in five stages, each adding one capability on top of a deterministic core.

Stage P0: deterministic single-chamber kernel.

- Implement semantic graph facts, rule facts, token-cost facts, and one context chamber.
- Implement enabled-rule search, token consumption, graph delta emission, and event logging.
- Require replay equivalence: snapshot plus seed plus budget produces the same chosen rules and event log.

Stage P1: evaluator-minted fuel.

- Add task evaluators and clamp-specific token minting.
- Add rent, redundancy tax, contradiction tax, and unsupported-claim tax.
- Prevent active somas from minting their own survival rewards.

Stage P2: ACS detection and causal replay.

- Scan event logs for recurrent closed rule/motif loops.
- Run paired replays: rule enabled versus disabled, ACS present versus ablated, context bridge present versus masked.
- Promote loops that have positive reward impact and positive metabolic surplus.

Stage P3: genomes, mutation, and reproduction.

- Reify useful ACSs as quoted rule-set genomes.
- Implement gated dequoting, structural mutation, recombination, and offspring endowment.

- Maintain lineage logs and compare child versus parent surplus.

Stage P4: coarse-grained PeTTa worker ecology.

- Run multiple context-chamber workers, ACS scans, replay batches, ECAN epochs, and reducers as PeTTa worker kinds.
- Use append-only event logs and reducers. No shared-hot-pool concurrent mutation.
- Add backend rendering tests from the same worker IR to PeTTa calls and future MeTTa-IL forms.

6.4 MeTTa-IL migration phase

Because the semantics live in portable facts, the MeTTa-IL migration should be a backend substitution, not an architectural rewrite. The correctness criterion is log equivalence: a small deterministic benchmark should satisfy

$$\text{Log}_{\text{PeTTa}}(W, \text{SNAP}, \text{seed}, \text{budget}) \equiv \text{Log}_{\text{MeTTaIL}}(W, \text{SNAP}, \text{seed}, \text{budget}) \quad (30)$$

that is, running the same worker W from the same snapshot, seed, and budget on either backend produces the same event log, modulo backend tags and harmless ID allocation differences.

To keep risk low, move worker families over in increasing order of difficulty, starting with workers that cannot corrupt shared state:

1. read-only replay and ablation workers;
2. deterministic context-chamber workers;
3. ACS scan workers;
4. evaluator and reducer workers;
5. mutation and reproduction workers;
6. multi-chamber par-groups.

Throughout, every worker, on either backend, must honor the same contract:

```
input:  worker facts, chamber snapshot, seed, budget, backend tag
run:    bounded local computation with no hidden global mutation
output: ordered event log, graph delta, token delta, summary facts
reduce: deterministic merge or explicit stochastic merge with recorded seed
```

The contract is what makes log equivalence checkable at all: a worker takes explicit inputs, does bounded local work with no hidden side effects, returns explicit outputs, and is merged in a recorded way.

7 Experiments and examples

The following five experiments are deliberately small and inspectable. Each one names a domain, the autocatalytic loops we expect to see emerge, the clamps that shape minting in that domain, and the kinds of tokens that successful behavior should earn. They are ordered roughly from a forgiving creative domain to stricter, higher-stakes ones.

7.1 Experiment 1: toy causal QA chamber

Domain: short texts parsed into semantic graphs, with simple questions such as “why did Maria drive to the store?” This is the simplest setting in which causal accountability can be tested end to end.

Expected ACSs:

```
question-type detector
+ causal-alignment rule
+ role-completion rule
+ paraphrase-collapse rule
+ answer-skeleton projection
+ compact explanation packer
```

Useful clamps:

- answer graph matches the question skeleton;
- answer uses a causal edge for why/how questions;
- explanation is compact;
- provenance reaches original sentence graph;
- unsupported bridge claims are penalized;
- redundant multi-path confidence inflation is taxed.

Rewards should mint answer-reward tokens, causal-inference tokens, compression tokens, and reproduction tokens for rule genomes that repeatedly solve variants of the same question type.

7.2 Experiment 2: memory-archipelago fiction chamber

Domain: a fictional world in which memories are transferable objects, can be stored in glass, traded, taxed, stolen, forged, or burned; collective memory stabilizes geography; and oaths are binding only while remembered. The interest of this domain is that its “world laws” can chain into nonobvious consequences, which is exactly what autocatalytic loops should surface.

Expected ACSs:

```
MemoryAsCurrency
-> MemoryTaxation
-> PoorPeopleSellAncestralMemories
-> CollectiveMemoryWeakens
-> GeographyDestabilizes
-> TradeRoutesFail
-> StateRaisesMemoryTaxes
-> MemoryAsCurrency
```

and

```
OathsRequireMemory
-> PoliticiansEraseOathMemories
-> LawsLoseBindingForce
```

```

-> CourtsRequireMemoryArchives
-> ArchivesBecomePoliticalWeapon
-> RebelsAttackArchives
-> OathsRequireMemory

```

A high-value bridge ACS would combine memory taxation, oath memory, and collective geography to generate an event such as a tax amnesty that causes a harbor to move because citizens reclaim inconsistent fragments of the treaty memory. (A bridge ACS is valuable precisely because it links two otherwise separate loops into one surprising-but-lawful consequence.)

Useful clamps:

- event follows from explicit world laws;
- event creates future plot branching;
- event creates a character dilemma;
- event is surprising but not arbitrary;
- event uses at least two independent world-rule modules;
- trope-level continuations without world-law causality are penalized.

Rewards should mint story-world-law tokens, novelty tokens, plot-branching tokens, character-dilemma tokens, and compression tokens.

7.3 Experiment 3: constrained biomedical hypothesis chamber

Domain: a toy or constrained biomedical corpus about post-viral cognitive fatigue, brain fog, exertion response, neurovascular dysfunction, immune activation, mitochondrial stress, autonomic compensation, and trial design. This is a research-hypothesis benchmark, not medical advice or automatic clinical recommendation, and the clamps are written specifically to keep it that way.

Expected ACSs:

```

PersistentImmuneActivation
-> EndothelialDysfunction
-> MicrovascularPerfusionReduced
-> CognitiveEnergyBottleneck
-> ExertionAmplifiesSymptoms
-> AutonomicCompensation
-> BBBVulnerability
-> MicroglialPriming
-> NeuroimmuneFeedback
-> PersistentImmuneActivation

```

Also look for ACSs that connect mechanism loops to subtype markers and trial constraints, which is what makes a hypothesis testable rather than merely narrative:

```

MechanismLoop
-> CandidateSubtype
-> MarkerSet
-> TrialEndpoint

```


-> FalsifiablePrediction
-> EvidenceUpdate
-> MechanismLoop

Useful clamps:

- hypothesis explains multiple symptom clusters;
- hypothesis supplies subtype markers;
- hypothesis states falsifiable predictions;
- hypothesis includes trial-design constraints;
- hypothesis avoids unsupported cure claims;
- hypothesis survives ablation better than isolated one-hop mechanisms;
- mechanism-to-marker-to-trial chain is compact.

Rewards should mint biomed-mechanism tokens, causal-abduction tokens, trial-design tokens, provenance tokens, and reproduction tokens. Penalize overconfident treatment claims, vague mechanism buzzwords, source-poor links, and hypotheses that cannot be operationalized.

7.4 Experiment 4: proof-state chemistry

Domain: small theorem-proving tasks represented as proof-state graphs. Here the interesting question is whether the chemistry discovers reusable proof *strategies* rather than one-off derivations.

Expected ACSs:

goal-shape detector
+ lemma-suggestion rule
+ algebraic-normalization rule
+ subgoal-splitting rule
+ proof-compression rule
+ replay-success macro-rule

Useful clamps:

- proof state distance to target theorem decreases;
- generated lemma is reusable across related tasks;
- proof term is shorter or more canonical after compression;
- branch explosion is penalized;
- successful proof traces mint tokens for the rule genomes that produced transferable proof macros.

This is a good domain for testing whether mortal semantic ACSs discover transferable inference-control strategies rather than one-off derivations, because the clamps explicitly reward reuse and compression over single-shot success.

7.5 Experiment 5: software bug-pattern chemistry

Domain: code graphs, test failures, static-analysis warnings, and repair patches. This is the strictest domain, because correctness is externally checkable by running tests.

Expected ACSs:

```
error-message motif
-> suspicious code motif
-> candidate invariant
-> patch schema
-> test-pass evidence
-> macro repair rule
-> error-message motif family
```

Useful clamps:

- patch passes tests;
- patch is minimal;
- patch preserves type and API constraints;
- patch explanation identifies the bug pattern;
- repair macro transfers to related failures;
- reward is withheld from patches that merely overfit tests.

This experiment tests whether the chemistry can evolve executable repair strategies while mortal computation prevents high-draw speculative patching from taking over, that is, whether the metabolic budget keeps wild guessing in check while still allowing genuine repair macros to emerge.

8 What to measure

Task metrics alone are insufficient for evaluating this kind of system. The chemistry’s internal ecology is also an object of study in its own right, and we care as much about *how* answers are produced as about whether they are correct. Measure at least:

- number of active semantic ACSs;
- number of mortal semantic ACSs with positive surplus;
- ablation impact of each ACS;
- recurrence of useful motifs across tasks;
- transfer of ACS genomes to new contexts;
- reward per CPU millisecond;
- reward per typed token class;
- lineage improvement from parent to child genomes;

- frequency of starvation and cause of death;
- redundancy tax paid by overactive loops;
- cross-context commutator defects;
- replay reproducibility under snapshot plus seed plus budget;
- PeTTa versus MeTTa-IL event-log equivalence after migration.

Underneath all of these individual metrics lies one decisive scientific question:

Do useful, causally influential, heritable semantic ACSs emerge, and do they improve when selected by token-gated metabolic pressure?

9 Risks and design rules

The following rules are stated as prohibitions because each one guards against a specific failure mode that the rest of the design would otherwise admit. They should be treated as invariants of any implementation.

1. Do not let active somas mint their own reward fuel. (Otherwise loops fund their own survival regardless of usefulness: semantic cancer.)
2. Do not allow PeTTa threads to mutate one chamber hot pool concurrently. (Otherwise replayability is lost.)
3. Do not make PeTTa-private predicates architectural primitives. (Otherwise the MeTTa-IL backend cannot reproduce the semantics.)
4. Do not globally forbid loops; require loops to pay rent and pass intervention tests. (Otherwise the system loses the redundancy that creativity needs.)
5. Do not use a single scalar energy budget when typed token signatures are needed for inference control. (Otherwise distinct reasoning resources cannot be throttled independently.)
6. Do not reward mere novelty without coherence, provenance, and task connection. (Otherwise the system optimizes for noise.)
7. Do not reward biomedical hypotheses that sound confident but lack falsifiable markers or trial constraints. (Otherwise it produces persuasive but untestable claims.)
8. Do not start with open-domain unrestricted generation. Start with small chambers where ACSs, rewards, and replay can be inspected. (Otherwise nothing can be debugged.)

10 Conclusion

The two ingredients answer complementary questions. Semantic Chemistry asks which semantic reaction networks become active, self-sustaining, useful, novel, and transferable. Mortal Computation asks what happens when every rewrite is costed, active processes can starve, quoted programs act as genomes, and reproduction has a measurable metabolic ledger. Their combination yields

Mortal Semantic Chemistry: a rule soup in which semantic inference strategies are *living objects*, subject to selection, rather than static procedures chosen by hand.

The implementation route is deliberately undramatic. Use PeTTa now as the orchestration layer over MORK/MM2 chambers, with deterministic event logs, evaluator-minted fuel, ACS scans, causal replay, and genome-level mutation. Represent every durable concept as portable worker IR facts. Later, migrate worker families to MeTTa-IL by requiring equivalent logs and reducer outputs, so that the migration is a swap of executor and not a redesign.

The practical payoff is an experimental system that can discover and evolve semantic inference-control organisms: QA loops, fictional world-law loops, biomedical mechanism loops, proof-strategy loops, and repair-strategy loops. The theoretical payoff is a new object of study: the mortal causal semantic ACS, a self-sustaining rule ecology that survives only by producing outputs with measurable causal and semantic value.

References

- [1] Ben Goertzel and GPT-5.5 Pro. *Semantic Chemistry: Algorithmic Chemistry on Semantic Graphs as a Creative Inference Control Strategy*. Draft manuscript, May 7, 2026.
- [2] L. G. Meredith. *Mortal Computation: Mortality, the Genome, and Life-History in Cost-Accounted GSLTs*. Working note, May 31, 2026.
- [3] Ben Goertzel and GPT-5.5 Pro. Portable worker IR and PeTTa-to-MeTTa-IL orchestration notes. Draft implementation notes, May 6, 2026.